```
/*
 * File:   main.cpp
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 24 de noviembre de 2023, 09:11 AM
 */

#include <iostream>
#include <fstream>
#include <iomanip>

using namespace std;

#include "Articulo.h"
#include "ArticuloPedido.h"
#include "Vendedor.h"

#include "funciones.h"

#define TAM_VENDEDORES 150
#define TAM_ARTICULOS 200

int main(int argc, char** argv) {
    struct Articulo arrArticulos[TAM_ARTICULOS]{};
    struct Vendedor arrVendedores[TAM_VENDEDORES]{};

    int cantArticulos,cantVendedores;

    cargarArticulos(arrArticulos,cantArticulos);
    mostrarArticulos(arrArticulos,cantArticulos,"PruebaArticulos.txt");
    cargarVendedores(arrVendedores,cantVendedores);
    mostrarVendedores(arrVendedores,cantVendedores,"PruebaVendedores.txt");
    actualizarVentas(arrArticulos,cantArticulos,arrVendedores,cantVendedores);
    mostrarArticulos(arrArticulos,cantArticulos,"ReporteArticulos.txt");
    mostrarVendedores(arrVendedores,cantVendedores,"ReporteVentas.txt");
    return 0;
}

/*
 * File:   Articulo.h
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 24 de noviembre de 2023, 09:12 AM
 */

#ifndef ARTICULO_H
#define ARTICULO_H

struct Articulo{
    char *codigo;
    char *descripcion;
    int stock;
    double precio;
    int cantVendidos;
    double montoVendido;
    int cantPerdidos;
    double montoPerdido;
};

#endif /* ARTICULO_H */
/*
 * File:   ArticuloPedido.h
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 24 de noviembre de 2023, 09:15 AM
 */
```

```
70
71  #ifndef ARTICULOPEDIDO_H
72  #define ARTICULOPEDIDO_H
73
74  struct ArticuloPedido{
75       char *codigo;
76       int cantidad;
77       int fechaUltPed;
78       double montoTotal;
79  };
80
81  #endif /* ARTICULOPEDIDO_H */
82
83  /*
84   * File:   Vendendor.h
85   * Author: Silvia
86   * Silvia Vargas Cáceres
87   *
88   * Created on 24 de noviembre de 2023, 09:16 AM
89   */
90
91  #ifndef VENDENDOR_H
92  #define VENDENDOR_H
93
94  struct Vendedor{
95      int codigo;
96      char *nombre;
97      double porcVentas;
98      double cuotaMin;
99      struct ArticuloPedido *articulosVendidos;
100     int cantArtVendidos;
101     struct ArticuloPedido *articulosPerdidos;
102     int cantArtPerdidos;
103     double montoTotalVendido;
104     double montoTotalPerdido;
105 };
106
107 #endif /* VENDENDOR_H */
108
109 /*
110  * File:   funciones.h
111  * Author: Silvia
112  * Silvia Vargas Cáceres
113  *
114  * Created on 24 de noviembre de 2023, 09:35 AM
115  */
116
117 #ifndef FUNCIONES_H
118 #define FUNCIONES_H
119
120 void cargarArticulos(struct Articulo *,int &);
121 void mostrarArticulos(struct Articulo *,int ,const char *);
122 void cargarVendedores(struct Vendedor *,int &);
123 void modificarNombre(char *);
124 void mostrarVendedores(struct Vendedor *,int ,const char *);
125 void mostrarArticulosVent(struct ArticuloPedido *,int ,ofstream &,const char *);
126 void actualizarVentas(struct Articulo *,int ,struct Vendedor *,int );
127 void procesarVenta(char *,int ,int ,int ,struct Articulo *,int ,
128         struct Vendedor *,int );
129 int buscarArticulo(char *,struct Articulo *,int );
130 int buscarVendedor(int ,struct Vendedor *,int );
131 void actualizarVentas(struct Articulo &,int ,struct ArticuloPedido* ,int &,
132         double &,int );
133 int buscarArticuloVendedor(struct ArticuloPedido * ,int ,char *);
134 void actualizarNoVentas(struct Articulo &,int ,struct Vendedor &,int );
135 char *leeCadenaExactaDelim(ifstream &,char );
136 void escribirSimbolo(ofstream &,char car,int );
137 #endif /* FUNCIONES_H */
138
```

```
139
140 /*
141  * File:   funciones.cpp
142  * Author: Silvia
143  * Silvia Vargas Cáceres
144  *
145  * Created on 24 de noviembre de 2023, 09:35 AM
146  */
147
148 #include <iostream>
149 #include <fstream>
150 #include <iomanip>
151
152 using namespace std;
153
154 #include <cstring>
155
156 #include "Articulo.h"
157 #include "ArticuloPedido.h"
158 #include "Vendedor.h"
159
160 #include "funciones.h"
161
162 #define TAM_ARTICULOS_VENDIDOS 50
163 #define NO_ENCONTRADO -1
164
165 void cargarArticulos(struct Articulo *arrArticulos,int &cantArticulos){
166     ifstream archArticulos("articulos.csv",ios::in);
167     if (not archArticulos.is_open()){
168         cout<<"Error: el archivo articulos.csv no se puede abrir "<<endl;
169         exit(1);
170     }
171     char *ptr;
172     cantArticulos=0;
173     char car;
174     while(true){
175         ptr=leeCadenaExactaDelim(archArticulos,',');
176         if (ptr==nullptr) break;
177         arrArticulos[cantArticulos].codigo=ptr;
178         arrArticulos[cantArticulos].descripcion=leeCadenaExactaDelim(archArticulos,',');
179         archArticulos>>arrArticulos[cantArticulos].stock>>car>>
180                 arrArticulos[cantArticulos].precio;
181
            arrArticulos[cantArticulos].cantVendidos=arrArticulos[cantArticulos].cantPerdidos
            =0;
182
            arrArticulos[cantArticulos].montoVendido=arrArticulos[cantArticulos].montoPerdido
            =0;
183         archArticulos.get();
184         cantArticulos++;
185     }
186 }
187
188 void mostrarArticulos(struct Articulo *arrArticulos,int cantArticulos,
189         const char *nombArch){
190     ofstream archReporte(nombArch,ios::out);
191     if (not archReporte.is_open()){
192         cout<<"Error: el archivo "<<nombArch<<" no se puede abrir "<<endl;
193         exit(1);
194     }
195     archReporte<<setprecision(2)<<fixed;
196     archReporte<<setw(50)<<"LISTADO DE ARTICULOS"<<endl;
197     archReporte<<"CODIGO"<<setw(20)<<"DESCRIPCION"<<setw(53)<<"STOCK"<<
198             setw(9)<<"PRECIO"<<setw(12)<<"VENDIDOS"<<setw(17)<<"MONTO VENDIDOS"<<
199             setw(15)<<"NO VENDIDOS"<<setw(20)<<"MONTO NO VENDIDOS"<<endl;
200     for (int i=0;i<cantArticulos;i++){
201         archReporte<<left<<arrArticulos[i].codigo<<' '<<setw(60)
202                 <<arrArticulos[i].descripcion
203                 <<right<<setw(10)<<arrArticulos[i].stock<<setw(10)
```

```
                <<arrArticulos[i].precio<<setw(6)<<arrArticulos[i].cantVendidos
                <<setw(15)<<arrArticulos[i].montoVendido
                <<setw(17)<<arrArticulos[i].cantPerdidos
                <<setw(15)<<arrArticulos[i].montoPerdido<<endl;
    }
}

void cargarVendedores(struct Vendedor *arrVendedores,int &cantVendedores){
    ifstream archVendedores("vendedores.csv",ios::in);
    if (not archVendedores.is_open()){
        cout<<"Error: el archivo vendedores.csv no se puede abrir "<<endl;
        exit(1);
    }
    int cod;
    cantVendedores=0;
    char car;
    while(true){
        archVendedores>>cod;
        if (archVendedores.eof()) break;
        arrVendedores[cantVendedores].codigo=cod;
        archVendedores.get();
        arrVendedores[cantVendedores].nombre=leeCadenaExactaDelim(archVendedores,',');
        archVendedores>>arrVendedores[cantVendedores].porcVentas>>car>>
                arrVendedores[cantVendedores].cuotaMin;
        arrVendedores[cantVendedores].articulosVendidos=new struct
        ArticuloPedido[TAM_ARTICULOS_VENDIDOS];
        arrVendedores[cantVendedores].cantArtVendidos=0;
        arrVendedores[cantVendedores].montoTotalVendido=0;
        arrVendedores[cantVendedores].articulosPerdidos=new struct
        ArticuloPedido[TAM_ARTICULOS_VENDIDOS];
        arrVendedores[cantVendedores].cantArtPerdidos=0;
        arrVendedores[cantVendedores].montoTotalPerdido=0;
        cantVendedores++;
    }
}

void mostrarVendedores(struct Vendedor *arrVendedores,int cantVendedores,
        const char *nombArch){
    ofstream archReporte(nombArch,ios::out);
    if (not archReporte.is_open()){
        cout<<"Error: el archivo "<<nombArch<<" no se puede abrir "<<endl;
        exit(1);
    }
    archReporte<<setprecision(2)<<fixed;
    archReporte<<setw(70)<<"LISTADO DE VENDEDORES"<<endl<<endl;
    for (int i=0;i<cantVendedores;i++){
        archReporte<<setw(12)<<"CODIGO"<<setw(20)<<"NOMBRE"<<setw(36)<<"% VENTAS"<<
            setw(14)<<"CUOTA MINIMA"<<setw(15)<<"TOTAL VENDIDO"<<
            setw(20)<<"TOTAL NO VENDIDO"<<endl;
        archReporte<<setw(4)<<i+1<<")"<<left<<setw(10)<<arrVendedores[i].codigo<<'
        '<<setw(43)
                <<arrVendedores[i].nombre<<right<<setw(10)
                <<arrVendedores[i].porcVentas<<setw(13)<<arrVendedores[i].cuotaMin
                <<setw(12)<<arrVendedores[i].montoTotalVendido
                <<setw(17)<<arrVendedores[i].montoTotalPerdido<<endl;
        mostrarArticulosVent(arrVendedores[i].articulosVendidos,
                arrVendedores[i].cantArtVendidos,archReporte,"VENDIDOS   ");
        mostrarArticulosVent(arrVendedores[i].articulosPerdidos,
                arrVendedores[i].cantArtPerdidos,archReporte,"NO VENDIDOS");
        double pagoSinCuotaMin,pagoVend=0,pagoCompleto;
        pagoSinCuotaMin=(arrVendedores[i].montoTotalVendido-arrVendedores[i].cuotaMin)*
                        arrVendedores[i].porcVentas/100;

        pagoCompleto=(arrVendedores[i].montoTotalVendido+arrVendedores[i].montoTotalPerdi
        do-
                arrVendedores[i].cuotaMin)*arrVendedores[i].porcVentas/100;
        if (pagoSinCuotaMin<0)
            pagoSinCuotaMin=0;
        if (pagoCompleto<0)
```

```
268             pagoCompleto=0;
269         archReporte<<"    TOTAL PAGADO AL
            VENDEDOR:                                                  "
270                 <<setw(12)<<pagoSinCuotaMin<<endl;
271         archReporte<<"    TOTAL PAGADO AL VENDEDOR SI SE HUBIERAN VENDIDO TODOS LOS
            PRODUCTOS: "
272                 <<setw(12)<<pagoCompleto<<endl;
273         archReporte<<"    TOTAL PERDIDO POR FALTA SE
            STOCK:                                    "
274                 <<setw(12)<<pagoCompleto-pagoSinCuotaMin<<endl<<endl;
275     }
276 }
277
278 void mostrarArticulosVent(struct ArticuloPedido *arrArticulos,int cantArt,
279         ofstream &archReporte,const char *tipoArt){
280     archReporte<<setw(15)<<"ARTICULOS "<<tipoArt<<endl;
281     archReporte<<setw(15)<<"CODIGO"<<setw(15)<<"CANTIDAD"<<setw(10)<<"FECHA"<<
282             setw(10)<<"MONTO"<<endl;
283     double total=0;
284     for (int i=0;i<cantArt;i++){
285         archReporte<<setw(7)<<i+1<<") "<<setw(6)<<arrArticulos[i].codigo
286
                    <<setw(10)<<arrArticulos[i].cantidad<<setw(16)<<arrArticulos[i].fechaUltP
                    ed
287                 <<setw(10)<<arrArticulos[i].montoTotal<<endl;
288         //total+=arrArticulos[i].montoTotal;
289     }
290     //archReporte<<setw(31)<<"MONTO TOTAL ARTICULOS "<<left<<tipoArt<<":
        "<<right<<setw(12)<<total<<endl;
291 }
292
293
294 void actualizarVentas(struct Articulo *arrArticulos,int cantArticulos,
295         struct Vendedor *arrVendedores,int cantVendedores){
296     ifstream archVentas("ventas.csv",ios::in);
297     if (not archVentas.is_open()){
298         cout<<"Error: el archivo ventas.txt no se puede abrir "<<endl;
299         exit(1);
300     }
301     int codVen,cant,dd,mm,aa,fecha;
302     char codArt[8],car;
303     while(true){
304         archVentas>>codArt;
305         if (archVentas.eof()) break;
306         archVentas>>codVen>>cant>>dd>>car>>mm>>car>>aa;
307         fecha=aa*10000+mm*100+dd;
308         procesarVenta(codArt,codVen,cant,fecha,arrArticulos,cantArticulos,
309                 arrVendedores,cantVendedores);
310     }
311 }
312
313 void procesarVenta(char *codArt,int codVen,int cant,int fecha,struct Articulo
    *arrArticulos,
314         int cantArticulos,struct Vendedor *arrVendedores,int cantVendedores){
315     int idPosArt,idPosVend;
316     idPosArt=buscarArticulo(codArt,arrArticulos,cantArticulos);
317     idPosVend=buscarVendedor(codVen,arrVendedores,cantVendedores);
318     if (idPosArt!=NO_ENCONTRADO and idPosVend!=NO_ENCONTRADO){
319         if (arrArticulos[idPosArt].stock>=cant){
320             actualizarVentas(arrArticulos[idPosArt],cant,
321                     arrVendedores[idPosVend].articulosVendidos,
322                     arrVendedores[idPosVend].cantArtVendidos,
323                     arrVendedores[idPosVend].montoTotalVendido,fecha);
324         }
325         else
326             actualizarVentas(arrArticulos[idPosArt],cant,
327                     arrVendedores[idPosVend].articulosPerdidos,
328                     arrVendedores[idPosVend].cantArtPerdidos,
329                     arrVendedores[idPosVend].montoTotalPerdido,fecha);
```

```
    }
}

int buscarArticulo(char *codArt,struct Articulo *arrArticulos,int cantArticulos){
    for (int i=0;i<cantArticulos;i++)
        if (strcmp(codArt,arrArticulos[i].codigo)==0)
            return i;
    return NO_ENCONTRADO;
}

int buscarVendedor(int codVen,struct Vendedor *arrVendedores,int cantVendedores){
    for (int i=0;i<cantVendedores;i++)
        if (codVen==arrVendedores[i].codigo)
            return i;
    return NO_ENCONTRADO;
}

void actualizarVentas(struct Articulo &datArticulo,int cantVen,
        struct ArticuloPedido* arrArticulos,int &cantArticulos,double &monto,int fecha){
    double montoCalc=cantVen*datArticulo.precio;
    int idArticulo=buscarArticuloVendedor(arrArticulos,cantArticulos,datArticulo.codigo);
    if (idArticulo==NO_ENCONTRADO){
        arrArticulos[cantArticulos].cantidad=0;
        arrArticulos[cantArticulos].montoTotal=0;
        arrArticulos[cantArticulos].codigo=new char[strlen(datArticulo.codigo)+1];
        strcpy(arrArticulos[cantArticulos].codigo,datArticulo.codigo);
        arrArticulos[cantArticulos].fechaUltPed=0;
        idArticulo=cantArticulos;
        cantArticulos++;
    }
    arrArticulos[idArticulo].cantidad+=cantVen;
    arrArticulos[idArticulo].montoTotal+=montoCalc;
    monto+=montoCalc;
    if (fecha>arrArticulos[idArticulo].fechaUltPed)
        arrArticulos[idArticulo].fechaUltPed=fecha;
}

int buscarArticuloVendedor(struct ArticuloPedido * arrArticulosPedidos,int cant,
        char *codigo){
    for (int i=0; i<cant;i++)
        if (strcmp(arrArticulosPedidos[i].codigo,codigo)==0)
            return i;
    return NO_ENCONTRADO;
}

char *leeCadenaExactaDelim(ifstream &archivo,char delim){
    char cadena[500],*ptrCadena;
    int longitud;
    archivo.getline(cadena,500,delim);
    if (archivo.eof()) return nullptr;
    longitud=strlen(cadena);
    ptrCadena=new char[longitud+1];
    strcpy(ptrCadena,cadena);
    return ptrCadena;
}

void escribirSimbolo(ofstream &archivo,char car,int cantCar){
    for (int i=0;i<cantCar;i++)
        archivo.put(car);
    archivo<<endl;
}
```